■ SPRING BOOT – 50 Deep Interview Questions (3-Line Answers Each)

1. What happens internally when Spring Boot application starts?

SpringApplication.run() bootstraps the application and prepares the environment.

It creates and refreshes the ApplicationContext.

Finally, it applies auto-configuration and starts the embedded server.

2. How does Auto-Configuration work internally?

It uses @EnableAutoConfiguration and metadata from spring.factories.

Spring checks classpath dependencies and applies configuration conditionally.

@Conditional annotations decide which beans should load.

3. What is the role of SpringApplication class?

It initializes the Spring context and handles startup logic.

It sets up environment, listeners, and application arguments.

It also manages failure analysis and logging.

4. What is ApplicationContext?

It is the IoC container managing beans and dependencies.

It handles lifecycle, configuration, and event propagation.

It extends BeanFactory with advanced features.

5. Difference between BeanFactory and ApplicationContext?

BeanFactory provides basic dependency injection.

ApplicationContext adds AOP, events, and resource handling.

Spring Boot uses ApplicationContext internally.

6. What is @SpringBootApplication?

It combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.

It simplifies configuration setup.

It is usually placed in the main class.

7. How does component scanning work?

Spring scans packages for stereotype annotations.

Detected classes are registered as beans.

Scanning starts from main class package.

8. What are Spring Boot Starters?

Starters are dependency bundles for common features.

They reduce manual dependency configuration.

Example: web, data-jpa, security starters.

9. How does embedded Tomcat work?

Spring Boot packages Tomcat inside the jar.

It auto-starts during application startup.

No external server deployment required.

10. What is Spring Boot Actuator?

Actuator exposes monitoring endpoints like health and metrics.

It helps monitor application in production.

Custom endpoints can also be created.

11. What is Condition Evaluation Report?

It shows which auto-configurations were applied or skipped.

Useful for debugging startup issues.

Enabled using debug mode.

12. What is @ConfigurationProperties?

It binds external properties to Java objects.

Supports hierarchical configuration mapping.

Better for large configuration sets.

13. What are Profiles?

Profiles separate environment-specific configurations.

Examples include dev, test, and prod.

Activated using spring.profiles.active.

14. What is DevTools?

DevTools enables automatic restart during development.

Improves developer productivity.

Not recommended for production use.

15. What is @RestController?

It combines @Controller and @ResponseBody.

Returns JSON directly in REST APIs.

Commonly used for backend services.

16. Difference between @Controller and @RestController?

@Controller returns views like JSP or Thymeleaf.

@RestController returns JSON/XML responses.

Used in RESTful services.

17. What is Global Exception Handling?

Implemented using @ControllerAdvice.

Centralizes exception handling logic.

Improves code maintainability.

18. What is ResponseEntity?

Represents complete HTTP response.

Allows setting status codes and headers.

Provides full response control.

19. What is @Bean?

Used to manually declare a bean in configuration class.

Useful for third-party classes.

Gives control over bean creation.

20. What is Circular Dependency?

Occurs when two beans depend on each other.

Constructor injection fails in this case.

Setter injection may resolve it.

21. What is Spring Boot Logging?

Uses Logback by default.

Logging levels configured in properties file.

Supports custom log patterns.

22. What is Caching in Spring Boot?

Caching stores frequently used data in memory.

Enabled using @EnableCaching.

Improves performance significantly.

23. What is Spring Boot Security?

Spring Security provides authentication and authorization.

Supports JWT and OAuth2.

Protects APIs and endpoints.

24. What is external configuration order?

Spring loads properties in defined priority order.

Command-line arguments override application.properties.

Environment variables also supported.

25. What is Filter in Spring Boot?

Filter works at servlet level.

Intercepts requests before reaching controller.

Used for logging and security.

26. What is Interceptor?

Interceptor works at Spring MVC level.

Executes before and after controller methods.

Used for authentication and logging.

27. What is Spring Boot CLI?

Command-line tool for running Spring apps quickly.

Reduces boilerplate code.

Useful for rapid prototyping.

28. What is @EnableAutoConfiguration?

It enables Spring Boot auto-configuration feature.

Automatically configures beans based on dependencies.

Part of @SpringBootApplication.

29. What is FailureAnalyzer?

Provides detailed error analysis during startup failure.

Helps debug configuration problems.

Improves developer experience.

30. What is Lazy Initialization?

Beans are created only when needed.

Reduces startup time.

Enabled via spring.main.lazy-initialization.

31. What is CommandLineRunner?

Executes code after application startup.

Useful for initialization logic.

Runs once at startup.

32. What is ApplicationRunner?

Similar to CommandLineRunner.

Provides access to application arguments.

Used during startup phase.

33. What is Spring Boot Packaging?

Applications are packaged as executable jars.

Includes embedded server.

Can also be packaged as WAR.

34. What is HealthIndicator?

Custom health checks for Actuator.

Monitors DB or external services.

Improves system observability.

35. What is Micrometer?

Metrics collection library used by Actuator.

Exports metrics to monitoring systems.

Supports Prometheus and others.

36. What is WebFlux?

Reactive programming framework in Spring Boot.

Non-blocking and asynchronous.

Uses Project Reactor.

37. What is Blocking vs Non-Blocking?

Blocking waits for task completion.

Non-blocking continues execution asynchronously.

WebFlux supports non-blocking.

38. What is @Conditional?

Loads beans based on specific conditions.

Used in auto-configuration.

Improves flexibility.

39. What is Banner in Spring Boot?

Displays custom startup message.

Configured via banner.txt.

Purely cosmetic feature.

40. What is Actuator Security?

Actuator endpoints can be secured.

Integrated with Spring Security.

Prevents unauthorized access.

41. What is Bean Scope?

Defines lifecycle of bean.

Singleton is default scope.

Other scopes include prototype and request.

42. What is Prototype Scope?

Creates new instance each time requested.

Not managed fully by container.

Used for stateful beans.

43. What is Dependency Injection?

Objects are provided by container.

Reduces tight coupling.

Core principle of Spring.

44. Constructor vs Setter Injection?

Constructor injection ensures immutability.

Setter injection allows optional dependencies.

Constructor injection preferred.

45. What is Spring Boot Dev Profile?

Used for development environment configuration.

Loads dev-specific properties.

Activated via profile setting.

46. What is Custom Starter?

Custom dependency bundle for reusable config.

Created for organization-wide reuse.

Reduces repetitive setup.

47. What is Embedded Servlet Container Customization?

Tomcat settings can be customized programmatically.

Configured using WebServerFactoryCustomizer.

Used for port and thread tuning.

48. What is Thread Pool Configuration?

Controls concurrency in server.

Configured via application properties.

Important for performance tuning.

49. How does Spring Boot handle graceful shutdown?

Completes active requests before shutdown.

Configured using server.shutdown property.

Improves reliability.

50. What is Production Readiness in Spring Boot?

Achieved using Actuator, metrics, and logging.

Supports monitoring and health checks.

Designed for cloud-native deployment.